Next-Concept Models on a Laptop GPU: A Fair Benchmark Against Autoregressive Models, and What Their Speed Is Worth in RAG and Agents

Aymen Rahmani
Final-year Data Science Engineering student
Independent learning project `https://github.com/Aymenrahmanii/NCM`

Abstract

Next-Concept Modeling (NCM) generates text coarse to fine, predicting every position of a level in parallel, and is reported to be up to 31× faster than a diffusion language model. We ask what that speed means on commodity hardware and against the baseline most practitioners care about: a same-size autoregressive (AR) model with a KV cache. On a 6 GB RTX 3050 laptop GPU we reproduce the official NCM checkpoints' quality (XSum ROUGE-L 27.8 vs. 28.0 reported; Sudoku-hard 84.5% vs. 86.1%) and benchmark them against pretrained AR models of similar size under identical conditions. At batch size 1 NCM matches (XSum, 268 vs. 274 ms) or slightly beats (WMT14, 1.2×) the AR model and has a tighter tail latency; at batch size 32 it is 2.3–8.7× slower and uses 5–16× more GPU memory, and its quality is lower (ROUGE-L 27.8 vs. 33.5; BLEU 20.0 vs. 30.8). Used as tools around a 3B LLM, NCM's speed does not help: compressing retrieved passages cuts RAG accuracy from 87.5% to 10%, NCM translation of a German corpus yields 40% RAG accuracy vs. 70% for a small AR translator, and in a tool-calling agent the LLM takes 91–97% of the time. A further suite probes where NCM breaks or wins: input and output length, batch size up to out-of-memory, faithfulness, robustness, out-of-distribution Sudoku, a 3B general LLM, and agent reliability. The limits suite shows that NCM's speed niche on this GPU is narrow (outputs of 20–64 tokens at batch 1, bounded by its fixed output canvas); that the same canvas caps translations at about 23 words, so BLEU falls from 16.3 to 0.04 as inputs grow while the AR model's rises; that NCM repeats itself 3–7× more often; and that the Sudoku model fails just outside its training distribution (32% with five clues removed) and never declines an unsolvable puzzle. A zero-shot 3B LLM is 1.4–3.7× slower than NCM, worse on XSum and better on translation; inside an agent, its tool sequencing, not NCM, decides success. All code, outputs and per-example logs are released.

1 Introduction

Most of the cost of serving a language model is sequential decoding: an AR model runs one forward pass per output token. NCM [3] instead builds the output in a fixed number of levels, from a short "gist" to the full sequence, predicting all positions of a level in parallel, in the spirit of next-scale prediction for images [15]. The headline result is a 31× speedup, but it is measured against a *diffusion* LM that needs 128 or more denoising steps, on an H100, with 1024-token outputs.

Practitioners mostly compare against AR models, often on small GPUs, with short outputs, and increasingly use small models as tools inside retrieval-augmented generation (RAG) [7] and agent pipelines. We therefore ask four questions:

1. Is NCM faster than a same-size AR model on a laptop GPU, and at what quality?

2. What does "31×" mean in that setting?

3. Does NCM's speed matter inside RAG and agent pipelines driven by a 3B LLM?

4. Where are the limits: input and output length, batch size, faithfulness, robustness, generalization, and comparison with a general-purpose LLM?

This is a personal learning project, not a new method: its contribution is an independent, fully reproducible evaluation on one commodity GPU, with predictions written down before each benchmark was run and reported as right or wrong.

2 Background: Next-Concept Modeling

NCM uses a multiscale VQ-VAE tokenizer that encodes a text sequence into L levels of discrete codes of increasing length. A transformer generator predicts level ℓ in one pass, conditioned on all coarser levels and an input prefix (the article or source sentence), so generation takes L passes regardless of output length; the decoder of the VQ-VAE then maps the finest level back to tokens. We first built a 5.6M-parameter toy version from scratch to check the mechanism (Section 4), then used the official task checkpoints for summarization (XSum [10]), German→English translation (WMT14 [2]) and Sudoku.

3 Setup

Hardware and software. NVIDIA RTX 3050 Laptop GPU (6 GB), WSL2 Ubuntu 24.04, PyTorch 2.14 with CUDA 13. All numbers in this paper come from this one machine.

Models. NCM XSum (88M generator + 60M VQ-VAE) vs. BART-base fine-tuned on XSum [6] (139M); NCM WMT14 (120M + 144M) vs. opus-mt-de-en [16, 5] (74M). The LLM for RAG and agents is qwen2.5:3b [13], 4-bit, served by Ollama.

Fairness rules. Same GPU, precision (fp16 autocast), examples and seed; greedy decoding for every model; warm-up before timing; `torch.cuda.synchronize()` around the clock; only generation is timed; the AR baseline keeps its KV cache; the AR model reads the same 512-token article prefix as NCM. Quality: ROUGE [8] and sacreBLEU [11, 12] on 500 test examples per task.

Pre-registration. For every benchmark we committed predictions to the repository before running it, and report each as right or wrong.

4 Toy NCM: the mechanism works, the samples do not

A 5.6M-parameter NCM (8.6M tokenizer) trained on short stories reconstructs 17.0% of tokens from the coarsest level alone and 98.7% from all six levels, so the coarse-to-fine representation behaves as intended. Generating 64 tokens it is 9.3× faster than a 6.9M-parameter AR model with a KV cache at batch 1 and 3.0× at batch 128 (6 passes vs. 64). But its samples are far less fluent (GPT-2 perplexity 377 vs. 24.5 for AR; real text 13.2): parallel prediction within a level makes local coherence hard.

Table 1: 500 test examples per task, fp16, RTX 3050. Throughput in examples/s.

	XSum		WMT14 De→En	
	NCM	BART-base	NCM	opus-mt
parameters (incl. tokenizer)	149M	139M	264M	74M
quality (ROUGE-L / BLEU)	27.8	**33.5**	20.0	**30.8**
latency batch 1, p50 / p95 (ms)	268 / **325**	274 / 354	**242 / 303**	283 / 536
throughput batch 1	3.7	3.7	**4.0**	3.4
throughput batch 8	5.0	**19.6**	**17.8**	15.9
throughput batch 32	5.0	**43.0**	20.5	**47.6**
peak GPU memory, batch 32	3.4 GB	**0.65 GB**	3.8 GB	**0.24 GB**

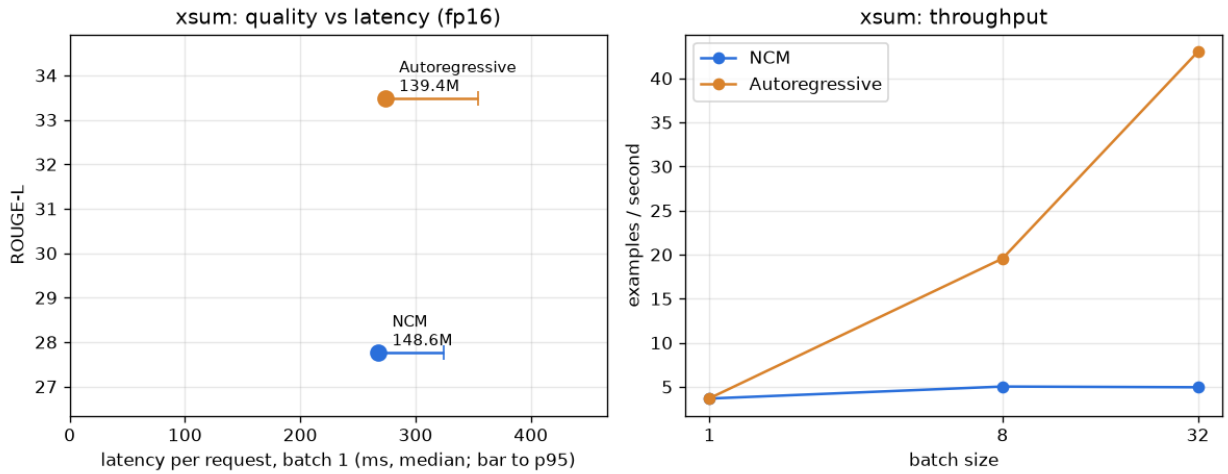


Figure 1: XSum: quality vs. latency at batch 1 (left) and throughput vs. batch size (right).

5 Reproducing the official checkpoints

On the first 50 test examples the official evaluation code and our wrapper agree to within 0.01 (XSum ROUGE-L 26.47 vs. 26.48; WMT14 BLEU 15.28 for both). On 500 examples we obtain XSum ROUGE-L 27.8 (paper: 28.0 on the full test set) and WMT14 BLEU 20.0 (paper: 22.6). Sudoku solve rates are 99.0% (easy), 97.5% (medium) and 84.5% (hard) on 200 puzzles each. fp16 is 1.9–2.5× faster than fp32 with the same quality. The official KV-cache generation path is 1.5× faster for WMT14 but drops BLEU from 20.0 to 3.8, so we do not use it.

6 NCM vs. same-size AR models

Table 1 and Figure 1 show the central result. At batch 1 NCM is as fast as BART on XSum and 1.2× faster than opus-mt on WMT14, with a tighter p95 because its number of passes does not depend on the output. At batch 32 the AR models are 2.3× (WMT14) to 8.7× (XSum) faster. Profiling explains why: each level pass re-runs the transformer over the full input prefix (512 tokens for XSum, 9 times), with no KV cache in this checkpoint's generation path, whereas BART encodes the article once and runs about 25 cheap cached decoder steps. At batch 1 the GPU is under-used either way; at batch 32 NCM's repeated prefix work dominates (7.6 s of 8.0 s per batch on XSum).

The quality gap mixes method and pretraining: both baselines start from large-scale pretraining,

while the NCM checkpoints are trained from scratch on the task data. The NCM paper's matched-data AR baseline scores ROUGE-L 24.4 (NCM 28.0) and BLEU 25.2 (NCM 22.6).

What "31×" means here. The figure compares NCM's 8–11 passes with 128–184 diffusion steps on 1024-token outputs. Against an AR model with a KV cache, on ∼20-word outputs, the step-count advantage mostly disappears.

Predictions vs. reality. We predicted a 2–5× batch-1 speedup (wrong: 1.0–1.2×), a shrinking advantage at larger batches (right, and it reverses), a larger advantage on XSum (wrong: XSum is NCM's worst case), and higher memory use (right).

7 NCM as a tool in RAG and agents

Setup. 40 XQuAD [1] questions over its 240 Wikipedia paragraphs, MiniLM embeddings [14] with FAISS [4], top-3 retrieval, qwen2.5:3b answering. An answer is correct if a gold answer string appears in it.

Context compression. Summarizing each retrieved paragraph before the LLM cuts accuracy from 87.5% to 10.0% (NCM) and 17.5% (BART) although retrieval still finds the right paragraph 97.5% of the time: a one-sentence news-style summary keeps the gist and drops the fact the question asks for. It is also slower (809 vs. 423 ms), because compression costs more than the shorter prompt saves.

Multilingual RAG. Translating the German version of the corpus (about 1,260 sentences) with NCM gives 40.0% accuracy vs. 70.0% with opus-mt (English originals: 87.5%), at about half the bulk speed (18.6 vs. 34.9 sentences/s at batch 32; paragraph BLEU 30.8 vs. 46.6). NCM translations replace entity names ("Luthers Schriften" → "Marx's writings"), which is fatal for retrieval. A first run with a sentence splitter that also split German ordinals ("am 3. Oktober") into separate inputs gave 37.5% and 67.5%; fixing the splitter helped both translators equally.

Agent. A hand-written tool-calling loop gives the LLM four tools: search and three NCM models (summarize, translate, solve Sudoku). Over six multi-step tasks the LLM accounts for 91–97% of wall time and NCM tools for 3–9%, so even an infinitely fast NCM would barely change end-to-end latency. Prompt design mattered more than speed: with a first prompt the LLM spent 28 s and 1,449 tokens solving the Sudoku itself instead of calling the tool.

8 Limits suite

Each experiment below pushes one axis until something breaks. Predictions are in `docs/limits.md`, committed before running.

8.1 Input length

Inputs are k consecutive WMT14 test sentences joined together, so length grows in a controlled way and the reference stays exact; the AR model is allowed up to 400 output tokens so its default cap is not what is measured. NCM's BLEU falls from 16.3 for single sentences to 10.1 for pairs, 2.6 for triples and 0.04 for eight sentences, while opus-mt *improves* from 24.7 to 31.5 (more context) and

Table 2: WMT14 De→En with k test sentences joined per input.

model	sentences_per_input	input_words	input_tokens	share_truncated	bleu	chrf	length_ratio
NCM (wmt14)	1	17.9	35.6	0.0	16.31	44.8	0.91
NCM (wmt14)	2	33.6	67.6	0.15	10.09	34.6	0.639
NCM (wmt14)	3	51.4	103.3	0.54	2.62	19.34	0.368
NCM (wmt14)	4	71.2	142.5	0.85	1.08	15.56	0.277
NCM (wmt14)	6	108.5	217.8	1.0	0.23	12.07	0.194
NCM (wmt14)	8	140.4	281.4	1.0	0.04	9.1	0.145
opus-mt-de-en	1	17.9	27.2	0.0	24.65	53.79	0.94
opus-mt-de-en	2	33.6	49.9	0.0	28.08	56.63	0.926
opus-mt-de-en	3	51.4	75.3	0.0	31.09	58.44	0.907
opus-mt-de-en	4	71.2	104.1	0.0	31.32	59.31	0.925
opus-mt-de-en	6	108.5	158.9	0.0	31.46	60.38	0.925
opus-mt-de-en	8	140.4	206.8	0.0	29.69	58.68	0.895

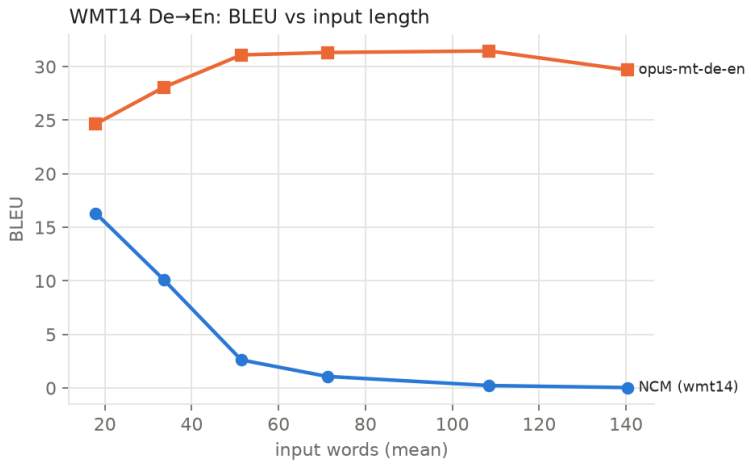


Figure 2: BLEU vs. input length on WMT14 De→En.

only dips to 29.7 at eight. The cause is not mainly the 96-token input window: at $k = 2$, 85% of inputs still fit in it, yet BLEU has already dropped by 38%. It is the fixed output canvas: NCM writes 18–25 words whatever the input (length ratio 0.91 at $k = 1$, 0.64 at $k = 2$, 0.15 at $k = 8$), so longer inputs are silently summarized rather than translated. Section 7 translated sentence by sentence for this reason.

8.2 Batch size up to (and past) GPU memory

On XSum, BART is faster at every batch size in this run (4.25 vs. 3.99 examples/s at batch 1, 10× at batch 32); NCM's throughput peaks at 6.7 examples/s at batch 16 and then *falls*, because each extra example adds another 512-token prefix to all nine passes. On WMT14, with a short prefix, NCM is faster at batch 2–8 (1.7× at batch 4) and opus-mt takes over from batch 16 for good, reaching 8.1× NCM's throughput at batch 128. Neither model ran out of memory: NCM's peak allocation exceeded the 6 GB card (6.8 GB on XSum, 9.4 GB on WMT14 at batch 128), because under WSL2 the Windows driver spills into shared system memory, which shows up as a throughput collapse (59 s per XSum batch of 128) rather than an error. Batch-1 times differ from Section 6 by up to 25% (e.g. opus-mt 203 vs. 283 ms) on different example orders and runs, so at batch 1 the two model families should be read as tied.

Table 3: Throughput and peak GPU memory by batch size (RTX 3050 6 GB, fp16).

task	model	batch_size	examples_per_s	ms_per_batch	peak_MB	oom
xsum	NCM (xsum)	1	3.99	250.7	1017	False
xsum	NCM (xsum)	2	5.43	368.6	1095	False
xsum	NCM (xsum)	4	5.92	675.6	1254	False
xsum	NCM (xsum)	8	6.42	1246.2	1565	False
xsum	NCM (xsum)	16	6.67	2399.7	2148	False
xsum	NCM (xsum)	32	6.34	5051.0	2740	False
xsum	NCM (xsum)	64	4.57	14003.1	4723	False
xsum	NCM (xsum)	128	2.17	59100.6	6779	False
xsum	bart-base-xsum	1	4.25	235.6	295	False
xsum	bart-base-xsum	2	6.78	294.8	306	False
xsum	bart-base-xsum	4	12.38	323.2	329	False
xsum	bart-base-xsum	8	23.29	343.5	374	False
xsum	bart-base-xsum	16	42.85	373.4	464	False
xsum	bart-base-xsum	32	64.47	496.3	644	False
xsum	bart-base-xsum	64	88.3	724.8	1005	False
xsum	bart-base-xsum	128	98.26	1302.7	1727	False
wmt14	NCM (wmt14)	1	4.3	232.5	1540	False
wmt14	NCM (wmt14)	2	8.32	240.3	1611	False
wmt14	NCM (wmt14)	4	15.75	254.0	1751	False
wmt14	NCM (wmt14)	8	20.46	391.0	2045	False
wmt14	NCM (wmt14)	16	25.7	622.6	2583	False
wmt14	NCM (wmt14)	32	27.76	1152.9	3630	False
wmt14	NCM (wmt14)	64	25.56	2504.0	5135	False
wmt14	NCM (wmt14)	128	24.24	5280.3	9442	False
wmt14	opus-mt-de-en	1	4.92	203.4	154	False
wmt14	opus-mt-de-en	2	5.98	334.7	156	False
wmt14	opus-mt-de-en	4	9.26	431.8	161	False
wmt14	opus-mt-de-en	8	18.79	425.7	172	False
wmt14	opus-mt-de-en	16	31.05	515.2	193	False
wmt14	opus-mt-de-en	32	58.48	547.2	236	False
wmt14	opus-mt-de-en	64	97.43	656.9	316	False
wmt14	opus-mt-de-en	128	196.31	652.0	482	False

8.3 Output length: where NCM's fixed step count pays off

The AR model is forced to write exactly L tokens and its latency is fitted as $a + bL$; NCM's cost is constant up to its canvas size. The break-even $L^* = (t_{\text{NCM}} - a)/b$ is NCM's speed niche on this GPU. This measures speed only: forced AR outputs are not meaningful text, and NCM cannot write beyond its canvas. AR decoding costs a fixed 17–38 ms plus 7.7–8.4 ms per token at batch 1; NCM costs a constant 209–219 ms. AR becomes slower than NCM beyond 20 output tokens (XSum) and 26 (WMT14), but NCM's canvas holds only 48 and 64 tokens, so its speed advantage exists only inside a window of 20–48 and 26–64 tokens, worth at most 2.1–2.3× at the canvas edge. At batch 8 the window disappears on XSum (break-even 104 tokens, beyond the canvas) and narrows to 33–64 tokens on WMT14. The step-count advantage that drives the 31× claim needs long outputs, which these checkpoints cannot produce.

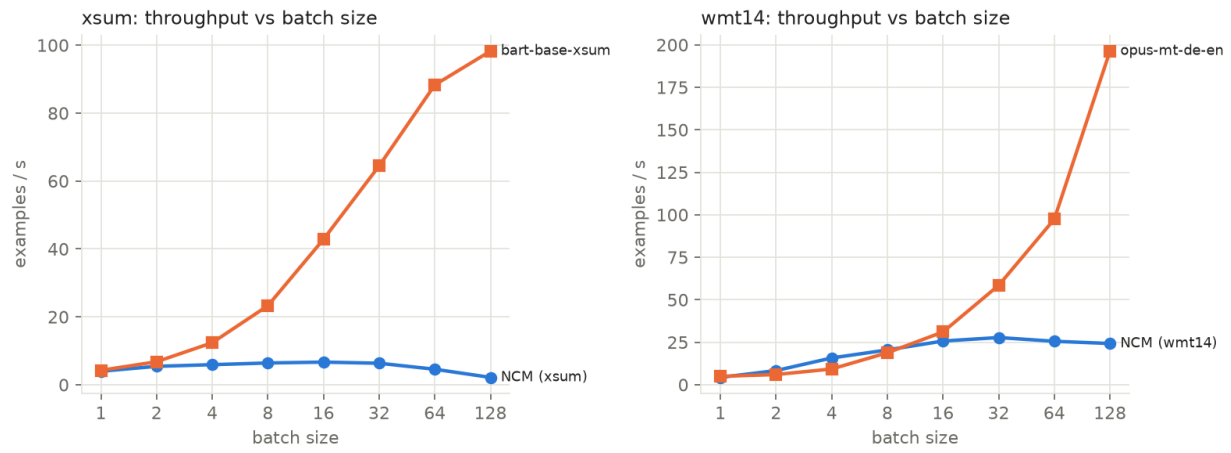


Figure 3: Throughput vs. batch size. Beyond 6 GB (NCM at batch 128) the driver spills into system memory instead of failing.

Table 4: Fitted AR latency (fixed + per-token) and the output length at which it exceeds NCM.

task	batch_size	ncm_ms	ar_ms_fixed	ar_ms_per_token	breakeven_tokens	ncm_canvas_tokens
xsum	1	209.3	37.7	8.376	20	48
xsum	8	982.0	29.7	9.165	104	48
wmt14	1	219.3	16.8	7.693	26	64
wmt14	8	318.3	-10.1	9.808	33	64

8.4 Faithfulness

Abstractive summarizers are known to hallucinate [9]. We count outputs that contain a number or a capitalized name absent from the source (for translation: absent from source and reference), and outputs with a repeated word or bigram. These proxies over-count equally for every model, so we compare models rather than read absolute rates. NCM repeats a word or bigram in 37.8% of XSum summaries (BART: 5.4%) and 36.4% of translations (opus-mt: 13.4%), consistent with parallel prediction within a level, where neighbouring positions cannot see each other's choice. It also introduces numbers absent from the source more often (XSum 20.2% vs. 11.4%; WMT14 3.8% vs. 0%) and, in translation, names absent from source and reference (21.4% vs. 7.4%). The name proxy is uninformative on XSum (56% vs. 51%), whose single-sentence style itself introduces entities.

8.5 Robustness to noisy input

Both model families are brittle in the same places. Uppercase input destroys both (XSum ROUGE-L 27.5→5.1 for NCM, 32.7→13.1 for BART; WMT14 BLEU 18.8→0.3 and 27.5→2.2). Lowercasing and removing punctuation cost NCM 4.9–6.9 points and the AR models 3.4–6.9; 10% typos cost both little. Absolute drops are similar, so relative to its lower starting point NCM loses more (lowercase WMT14: −31% vs. −22%).

8.6 Sudoku out of distribution

NCM vs. an exact backtracking solver and qwen2.5:3b on: the hard test set; the same puzzles with digits relabelled and the grid transposed (logically identical); 5 or 15 clues removed (any

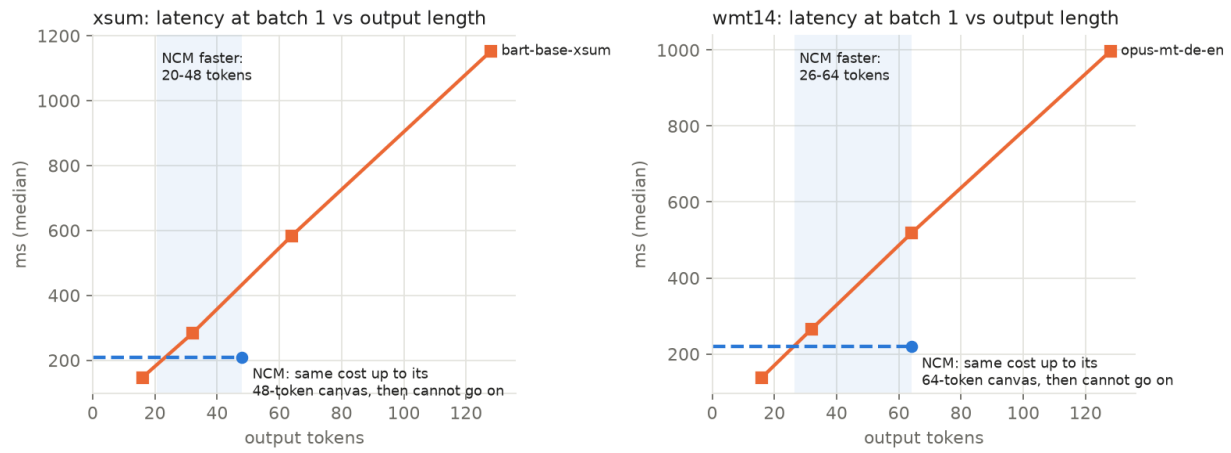


Figure 4: Latency at batch 1 vs. output length.

Table 5: Share of outputs with a number or name absent from the source, or a repetition.

task	model	n	share_invented_number	share_invented_name	share_repetition	share_any
xsum	NCM	500	0.202	0.562	0.378	0.802
xsum	AR baseline	500	0.114	0.512	0.054	0.588
wmt14	NCM	500	0.038	0.214	0.364	0.5
wmt14	AR baseline	500	0.0	0.074	0.134	0.196

valid completion counts); unsolvable puzzles (a duplicate digit in a row, where the right answer is to say so); and already-solved grids. NCM solves 84% of the hard test puzzles and 85% of the same puzzles relabelled and transposed, so it is invariant to these symmetries. But it collapses just outside the training distribution: 32% with five clues removed and 1% with fifteen, although these puzzles are *easier* to satisfy (any valid completion counts) and the exact solver completes 100% and 99%. Given an unsolvable puzzle it always returns a full grid, never signalling a problem; given an already-solved grid it overwrites 7–10 of the given digits in the examples we inspected and returns an invalid grid every time. Every puzzle in the hard test set has exactly 51 blanks, which suggests the model learned a fixed amount of "work" rather than the constraint of keeping the clues: removing clues or giving it none to fill moves the input outside anything it has seen. A simple backtracking solver on the CPU is exact and takes about 1 ms per hard puzzle, against NCM's 23–38 ms per puzzle on the GPU in batches. The general LLM shows the mirror image. On 20 puzzles per set, qwen2.5:3b solves none, returns a full 81-digit grid only 20–45% of the time, and never says a puzzle is unsolvable (it was asked to), taking 1.8–4.5 s per puzzle; but given an already-solved grid it returns it unchanged every time. The LLM can copy the clues and cannot solve; NCM can solve, in distribution, and cannot copy the clues.

8.7 A specialist vs. a general LLM

The same 100 examples per task through NCM, the AR baseline and zero-shot qwen2.5:3b, one request at a time, end-to-end latency. The two tasks split. On XSum, NCM (149M) beats the 3B LLM on ROUGE-L (27.6 vs. 20.2) and is 3.7× faster (265 vs. 968 ms median), because XSum rewards a learned house style the zero-shot LLM does not imitate; the trained BART is best on both counts (31.9). On WMT14 the LLM beats NCM on quality (BLEU 21.9 vs. 16.2; chrF 51.2

Table 6: Quality on perturbed inputs and the drop relative to clean input.

task	model	perturbation	rougeL	bleu	drop_vs_clean
xsum	NCM (xsum)	clean	27.53		0.0
xsum	NCM (xsum)	typos_10pct	27.47		0.06
xsum	NCM (xsum)	typos_30pct	24.28		3.25
xsum	NCM (xsum)	lowercase	22.64		4.89
xsum	NCM (xsum)	uppercase	5.12		22.41
xsum	NCM (xsum)	no_punctuation	21.21		6.32
xsum	bart-base-xsum	clean	32.69		0.0
xsum	bart-base-xsum	typos_10pct	31.83		0.86
xsum	bart-base-xsum	typos_30pct	29.34		3.35
xsum	bart-base-xsum	lowercase	28.42		4.27
xsum	bart-base-xsum	uppercase	13.14		19.55
xsum	bart-base-xsum	no_punctuation	29.29		3.4
wmt14	NCM (wmt14)	clean		18.76	0.0
wmt14	NCM (wmt14)	typos_10pct		15.76	3.0
wmt14	NCM (wmt14)	typos_30pct		14.11	4.65
wmt14	NCM (wmt14)	lowercase		12.94	5.82
wmt14	NCM (wmt14)	uppercase		0.28	18.48
wmt14	NCM (wmt14)	no_punctuation		11.84	6.92
wmt14	opus-mt-de-en	clean		27.5	0.0
wmt14	opus-mt-de-en	typos_10pct		24.75	2.75
wmt14	opus-mt-de-en	typos_30pct		21.74	5.76
wmt14	opus-mt-de-en	lowercase		21.58	5.92
wmt14	opus-mt-de-en	uppercase		2.21	25.29
wmt14	opus-mt-de-en	no_punctuation		20.59	6.91

vs. 44.7) at 1.4× NCM's latency, and the 74M opus-mt beats both (24.7 BLEU, 265 ms). A 20× larger general model is therefore not automatically better than a small specialist, but NCM is never the best of the three. Caveat: NCM's WMT14 latency in this run (377 ms median, 1334 ms p95) is higher than in the batch sweep (232 ms), most likely because the LLM stayed resident on the same 6 GB GPU after the XSum part; the LLM is also 4-bit quantized.

8.8 Agent reliability

Each Phase 5 task runs 10 times at temperature 0.7 (and once at 0). The agent's failures are systematic, not random: every task behaves the same in all 11 runs, so sampling does not rescue it. Lookups work (search: 100%; two questions: 90% at temperature 0.7). But in every run the LLM (i) skips the summarization step it was asked for (accuracy is unaffected, as the answer is in the passage); (ii) translates the German question and then answers from memory without searching, producing a confident, wrong answer (0%); (iii) calls the translator and the summarizer *in parallel* in a single turn, so the English-only summarizer receives German and the final answer is useless; and (iv) miscopies the 81-digit Sudoku (e.g. 83 cells) in 11 of 11 runs, which the tool rejects. None of these failures involves NCM's speed or quality: with a 3B planner, the bottleneck of a tool-using pipeline is the planner's ability to sequence tools and copy arguments.

Table 7: Correct = valid solution, or for the unsolvable set, saying so.

set	solver	n	correct	valid_solution	full_grid_returned	said_unsolvable	ms_per_puzzle
hard	exact solver (CPU)	100	1.0	1.0	1.0	0.0	1.03
shuffled	exact solver (CPU)	100	1.0	1.0	1.0	0.0	1.06
fewer_clues_5	exact solver (CPU)	100	1.0	1.0	1.0	0.0	3.74
fewer_clues_15	exact solver (CPU)	100	0.99	0.99	0.99	0.0	191.08
unsolvable	exact solver (CPU)	100	1.0	0.0	0.0	1.0	0.01
already_solved	exact solver (CPU)	100	1.0	1.0	1.0	0.0	0.05
hard	NCM sudoku-hard	100	0.84	0.84	1.0	0.0	28.28
shuffled	NCM sudoku-hard	100	0.85	0.85	1.0	0.0	31.53
fewer_clues_5	NCM sudoku-hard	100	0.32	0.32	1.0	0.0	38.06
fewer_clues_15	NCM sudoku-hard	100	0.01	0.01	1.0	0.0	38.5
unsolvable	NCM sudoku-hard	100	0.0	0.0	1.0	0.0	25.96
already_solved	NCM sudoku-hard	100	0.0	0.0	1.0	0.0	23.1
hard	LLM qwen2.5:3b	20	0.0	0.0	0.3	0.0	2653.26
shuffled	LLM qwen2.5:3b	20	0.0	0.0	0.45	0.0	4368.13
fewer_clues_5	LLM qwen2.5:3b	20	0.0	0.0	0.35	0.0	1977.45
fewer_clues_15	LLM qwen2.5:3b	20	0.0	0.0	0.2	0.0	1804.69
unsolvable	LLM qwen2.5:3b	20	0.0	0.0	0.4	0.0	4511.48
already_solved	LLM qwen2.5:3b	20	1.0	1.0	1.0	0.0	3250.52

Table 8: NCM, the AR baseline and qwen2.5:3b (zero-shot, 4-bit) on the same examples.

task	system	params_M	rougeL	bleu	chrf	output_words	latency_p50_ms	latency_p95_ms
xsum	NCM (xsum)	149	27.56			21.1	264.9	365.9
xsum	bart-base-xsum	139	31.87			18.6	331.2	528.6
xsum	qwen2.5:3b (zero-shot)	3090	20.24			23.8	968.3	1459.8
wmt14	NCM (wmt14)	264		16.15	44.7	18.3	377.3	1333.8
wmt14	opus-mt-de-en	74		24.67	53.82	18.9	265.3	488.1
wmt14	qwen2.5:3b (zero-shot)	3090		21.93	51.16	18.6	535.5	988.2

9 Discussion

On a laptop GPU and short outputs, NCM's parallel decoding buys predictable single-request latency, not throughput: its passes repeat work over the input prefix that an AR model with a KV cache does once. Its speed would matter for long outputs (Section 8), against diffusion models, or with a cached prefix, none of which the released task checkpoints exercise. As a tool in LLM pipelines, NCM's speed is dwarfed by the LLM, and its weaknesses (lower quality than pretrained baselines, invented entities) are exactly the ones RAG punishes. The limits suite sharpens this. NCM's real speed niche on this GPU is single requests with outputs of roughly 20–64 tokens; below that the AR model is as fast, above it NCM cannot write at all. Its fixed canvas also explains its failure modes: long inputs are compressed into ∼23 words, and a constraint solver trained on puzzles with a fixed number of blanks does not respect the clues once that number changes. Against a 3B general LLM, NCM is faster and better at a narrow, style-specific task (XSum) and worse at a general one (translation), and the LLM's own limits (copying long digit strings, sequencing tools) dominate any pipeline that uses NCM as a tool.

10 Limitations

One GPU and one machine; results on an H100 or with larger batches may differ. Baselines are pretrained while NCM checkpoints are not, so the quality gap is not only the method's. Evaluation sets are small (500 examples per task, 40 RAG questions, six agent tasks): enough to see large effects, not small ones. RAG accuracy uses lenient string containment. fp16 is autocast with fp32

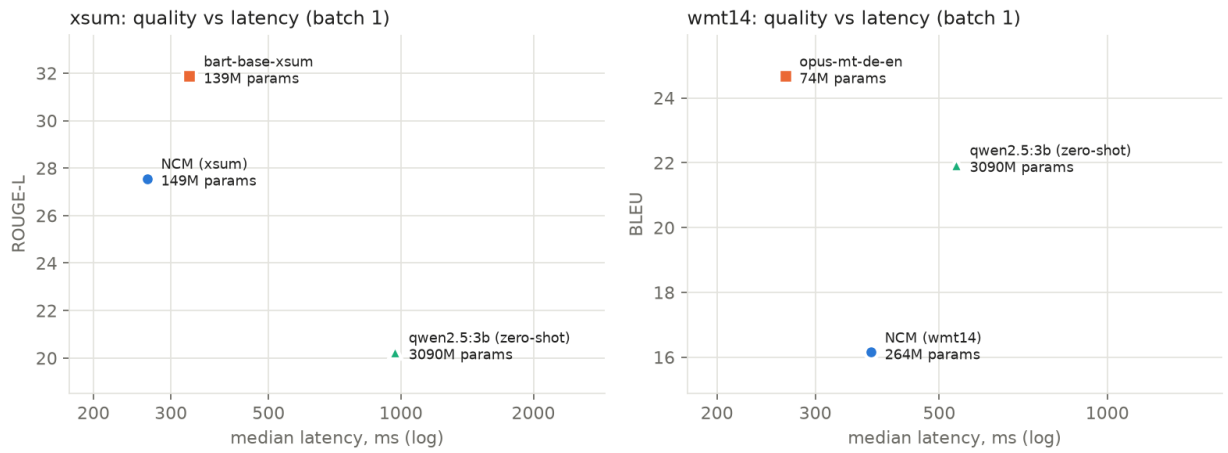


Figure 5: Quality vs. median latency at batch 1.

Table 9: Agent runs per task: share calling the needed tools, tool errors, and accuracy.

task	temperature	runs	right_tools	runs_with_tool_error	out_of_steps	accuracy	mean_llm_calls	mean_total_ms
search_answer	0.0	1	1.0	0.0	0.0	1.0	2.0	1716.6
search_answer	0.7	10	1.0	0.0	0.0	1.0	2.0	1463.7
search_summarize_answer	0.0	1	0.0	0.0	0.0	1.0	2.0	1738.7
search_summarize_answer	0.7	10	0.0	0.0	0.0	1.0	2.0	1762.9
translate_summarize	0.0	1	1.0	0.0	0.0		2.0	10778.8
translate_summarize	0.7	10	1.0	0.0	0.0		2.0	9791.3
german_question_rag	0.0	1	0.0	0.0	0.0	0.0	2.0	3801.6
german_question_rag	0.7	10	0.0	0.0	0.0	0.0	2.0	4473.5
sudoku	0.0	1	1.0	1.0	0.0		2.0	3365.0
sudoku	0.7	10	1.0	1.0	0.0		2.0	3620.2
two_questions	0.0	1	1.0	0.0	0.0	1.0	2.0	16879.6
two_questions	0.7	10	1.0	0.0	0.0	0.9	2.0	7640.3

weights. We did not re-measure the diffusion baseline behind the 31× claim. Faithfulness uses string proxies, not human judgment.

11 Reproducibility

Code, configuration, every output and per-example log are at `https://github.com/Aymenrahmanii/NCM`. `bash scripts/reproduce_results.sh` recreates every number; the service, tests and CI run without a GPU using stand-in models.

Use of AI assistance

Parts of the code, the documentation, the predictions for the limits suite (Section 8) and a first draft of this report were written with the help of an AI assistant (Claude, Anthropic). All experiments were run by the author on a personal laptop, every number is taken from the result files in the repository, and the author takes full responsibility for the content.

References

[1] Mikel Artetxe, Sebastian Ruder, and Dani Yogatama. On the cross-lingual transferability of monolingual representations. In *Proceedings of ACL*, 2020.

[2] Ondřej Bojar, Christian Buck, Christian Federmann, Barry Haddow, Philipp Koehn, Johannes Leveling, Christof Monz, Pavel Pecina, Matt Post, Herve Saint-Amand, Radu Soricut, Lucia Specia, and Aleš Tamchyna. Findings of the 2014 workshop on statistical machine translation. In *Proceedings of the Ninth Workshop on Statistical Machine Translation*, 2014.

[3] Hazy Research. Next-concept modeling, 2026. Code: `https://github.com/HazyResearch/ncm`. TODO: full author list.

[4] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.

[5] Marcin Junczys-Dowmunt, Roman Grundkiewicz, Tomasz Dwojak, Hieu Hoang, Kenneth Heafield, Tom Neckermann, Frank Seide, Ulrich Germann, Alham Fikri Aji, Nikolay Bogoychev, André F. T. Martins, and Alexandra Birch. Marian: Fast neural machine translation in C++. In *Proceedings of ACL 2018, System Demonstrations*, 2018.

[6] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of ACL*, 2020.

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

[8] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, 2004.

[9] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. On faithfulness and factuality in abstractive summarization. In *Proceedings of ACL*, 2020.

[10] Shashi Narayan, Shay B. Cohen, and Mirella Lapata. Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. In *Proceedings of EMNLP*, 2018.

[11] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of ACL*, 2002.

[12] Matt Post. A call for clarity in reporting BLEU scores. In *Proceedings of the Third Conference on Machine Translation*, 2018.

[13] Qwen Team. Qwen2.5 technical report, 2024.

[14] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, 2019.

[15] Keyu Tian, Yi Jiang, Zehuan Yuan, Bingyue Peng, and Liwei Wang. Visual autoregressive modeling: Scalable image generation via next-scale prediction. In *Advances in Neural Information Processing Systems*, 2024.

[16] Jörg Tiedemann and Santhosh Thottingal. OPUS-MT – building open translation services for the world. In *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*, 2020.